Differential Evolution Algorithm

Jonah Catlin

Goal: Want to find a minimizer for some function.

- We have discussed many different approaches
  - Gradient Descent
  - Newton's Method
  - Subgradient Descent
  - Etc.

- Note that all of these approaches require some sort of gradient information!

Metaheuristics

- What if we don’t have access to gradient information?
  - Objective function is not smooth (discrete steps)
  - Objective function is too complicated otherwise

- Metaheuristics are a class of models that can perform optimization to find near-optimal solutions to problems, while only having access to the objective function
  - As a tradeoff, metaheuristic methods typically cannot make any guarantees for convergence

Differential Evolution (DE)

- DE is an example of a metaheuristic method
  - What information is required?
    - Loss function
    - Search range for solution

- Overview
  - DE employs a randomly initialized “population” of points over a given search region
  - Iteratively “mutates” these points with each other to create new “candidates”
  - Keeps the fittest candidates, and discards others
    - “Fitness” is evaluated via the loss function

Hyperparameters for DE

- NP: Population Size
  - NP >= 4
    - The mutation step requires 1 candidate as the first “parent”
    - 3 more are used to construct a second “parent”
- CR: Crossover Probability
  - $CR \in [0, 1]$
    - This is the chance that any given “gene” from an existing candidate is “mutated”
- F: Differential Weight
  - $F \in [0, 2]$
    - This is essentially a measure of how much each “gene” is mutated by, when one occurs
- Typical Settings:
  - $NP = 10*n,\ CR = 0.9,\ F = 0.8$
    - n is dimension of problem

Algorithm: Initialization & Stopping Conditions

- The first step of the DE algorithm is to pick an initial population
- Sampling uniformly from the given search space, we pick NP initial candidates
  - This sampling is easy, since we can choose the search space
    - Usually a bounding box of some sort

- After initialization, we run the algorithm until a specified stopping condition is met
  - Maximum iterations
  - Improvement becomes slow enough
  - Etc.

Algorithm: Evolution Step

- This process is run once for each candidate in the current population, x:

- Select three other candidates: a, b, and c.
- Compute a new point y:
  - y = a + F * (b - c)
- Points x and y will be used as the two parents to create a new, “mutated” point z.
  - This will be done by choosing the either the value of x or y at each coordinate, using a specific rule

Algorithm: Evolution Step (Continued)

- Choose one coordinate randomly.
  - For this coordinate, $z_i = y_i$.
    - (This enforces that the candidate will not be identical to the first parent)
- For all other coordinates:
  - Generate a random number, r, between 0 and 1.
  - If r < CR, then let $z_i = y_i$.
  - If r >= Cr, then let $z_i = x_i$
- Next evaluate the two candidates x and z:
  - The point with a lower loss will be a part of the population in the next iteration
    - Note that this guarantees that the loss of the population will always decrease, across iterations.
- Once all candidates for the next iteration have been chosen, the population is updated as a block.

Demo

(Link to Colab Notebook)